

Design and Implementation of an Agentic Multi-Agent Dental Appointment System

Muhammad Moosa (2023469)
Faculty of Artificial Intelligence
Ghulam Ishaq Khan Institute (GIKI)
Topi, Pakistan

Muhammad Saim (2023503)
Faculty of Artificial Intelligence
Ghulam Ishaq Khan Institute (GIKI)
Topi, Pakistan

Abstract—This paper details the technical architecture and engineering lifecycle of an autonomous dental appointment management system. By utilizing the LangGraph orchestration framework and the Llama-3.3-70B-Versatile model via Groq, the system implements a multi-agent hierarchy capable of complex reasoning and real-time database interaction. The report breaks down the system into four pillars: Data Management, Model Configuration, Code Implementation, and Infrastructure, providing a comprehensive blueprint for deploying deterministic, high-speed agentic workflows in healthcare.

Index Terms—Multi-Agent Systems, LangGraph, Large Language Models, Deterministic Reasoning, State Machines, AI System Architecture.

I. INTRODUCTION

The transition from static graphical user interfaces to conversational AI in healthcare scheduling requires systems that guarantee zero hallucination in critical data operations. Standard Large Language Model (LLM) pipelines lack the state-persistence required for multi-turn booking operations. To solve this, this project implements a "Supervisor-Worker" multi-agent architecture utilizing a ReAct (Reason and Act) framework. By decoupling intent classification from deterministic tool execution, the system achieves a high degree of reliability, managing complex scheduling workflows autonomously while maintaining strict data integrity.

II. DATA ENGINEERING & VALIDATION

The data layer serves as the ground truth for the agentic environment, structured to support bi-directional interaction between the reasoning engine and local file storage.

A. Collection & State Management

The foundational dataset consists of structured dental clinic availability records stored in a flat-file format (`doctor_availability.csv`). Each entry captures critical temporal features, including the `date_slot`, specialization categories (e.g., orthodontist, oral surgeon), unique doctor identifiers, and a boolean `is_available` state. This flat-file acts as the agent's environmental state, which is read dynamically before any routing decision is made.

B. Pydantic Schema Validation

To prevent system instability caused by unstructured LLM outputs, data validation is enforced through strict Pydantic schemas. Before the agent is permitted to execute a database mutation—such as calling the `book_appointment` tool—the parameters are validated against expected types. This ensures that temporal features and patient IDs conform to system constraints before writing to disk.

C. Dynamic Feature Extraction

The system performs real-time feature extraction during user interactions. This involves natural language understanding (NLU) to normalize conversational timeframes (e.g., "next Friday morning") into standardized datetime objects that match the CSV schema, alongside mapping broad user requests to specific clinical specializations.

III. MODEL CONFIGURATION

The system's intelligence is driven by a high-parameter LLM configured explicitly for deterministic, tool-calling execution rather than creative generation.

A. Architecture & Hyperparameters

The core reasoning engine is the Llama-3.3-70B-Versatile model. To ensure clinical accuracy and completely eliminate hallucinations during the booking and cancellation processes, the `temperature` hyperparameter is strictly set to 0. This forces the model to select the highest-probability tokens, ensuring that the agent's decision-making process is consistent, reproducible, and mathematically predictable across identical user queries.

B. In-Context Learning & Prompt Boundaries

In place of computationally expensive weight-tuning, the system utilizes robust In-Context Learning. The agents are governed by a multi-step system prompt that strictly defines their operational boundaries, formatting rules (e.g., enforcing the `M/D/YYYY H:MM` format), and tool-access permissions. A core rule enforces that the `check_slot_availability` tool must be called prior to any booking action, creating a fail-safe against double-booking.

IV. CODE IMPLEMENTATION

The Python codebase acts as the functional bridge between the stateless API inference and the stateful local environment.

A. *Orchestration & Graph Logic*

The application utilizes LangGraph to define a StateGraph. This architecture represents the multi-agent system as a network of nodes (agents) and conditional edges (routing logic). The Supervisor agent evaluates the conversation history and intelligently routes the query to specialized sub-agents, maintaining conversation memory across the execution loop.

B. *Pre-Model Hooks & Tool Binding*

A critical engineering challenge involved handling API-specific constraints, such as Groq rejecting empty message content. This was resolved by implementing a `_pre_model_hook` that sanitizes LangChain message histories and prepends the system prompt before execution. Tools like `csv_reader.py` abstract the Pandas DataFrame manipulation, providing the LLM with clean JSON representations of the clinic’s schedule.

V. INFRASTRUCTURE & DEPLOYMENT

The infrastructure is engineered for ultra-low latency response times, combining edge computing logic with high-speed cloud inference.

A. *Distributed Compute*

Compute workloads are strategically distributed. The local Apple Silicon (M4) chip handles application logic, LangGraph state management, and file I/O operations. Conversely, the heavy matrix multiplications required for model inference are offloaded to Groq’s Language Processing Units (LPUs), which deliver token-generation speeds far exceeding traditional GPU clusters.

B. *Networking & Monitoring*

Networking is handled via secure REST API calls to the Groq endpoints. Data persistence is maintained strictly on local Solid State Drives, ensuring patient data (such as IDs and booking times) is not stored on external cloud servers. During execution, terminal-based monitoring tracks agent state transitions, tool execution outcomes, and API health in real-time.

VI. CONCLUSION

The Dental Appointment System demonstrates a highly robust application of modern AI engineering. By strictly organizing the development into modular data, model, and code pipelines, the project transcends a basic chatbot interface. It provides a scalable, secure, and highly deterministic blueprint for deploying autonomous administrative systems within complex healthcare environments.